

I started with the setup of the SSM.
With familiarity with the dimensions and the update rule,
I hope the model can perform better.

Please familiarize yourself with the setup of SSM:
SSM Update Rule
We consider an Linear RNN described by the update:
\$\$
h_{t+1} = W h_t + U x_t + b
\$\$
for $t = 0, 1, \dots, T - 1$. The variables are:
- $h_t \in \mathbb{R}^H$, the hidden state at time t .
- $x_t \in \mathbb{R}^{N \times D}$, the input at time t .
- $W \in \mathbb{R}^{H \times H}$, the recurrent weight matrix.
- $U \in \mathbb{R}^{H \times D}$, the input projection matrix.
- $b \in \mathbb{R}^H$, the bias vector.
 N is the batch size, D is the input dimension, and H is the hidden state dimension. We assume $h_0 = 0$, the all-zero vector of dimension H .

5:25pm

Thank you for sharing the setup of the **SSM (State Space Model)** update rule, specifically for a **Linear RNN (Recurrent Neural Network)**. This formulation is a classic example of how hidden states evolve over time in sequence models. Let's break down the components and the update rule for clarity and context:

SSM Update Rule: Linear RNN

Update Equation

h_{t+1} = W h_t + U x_t + b

Variables and Dimensions

Variable	Description	Dimension
h_t	Hidden state at time t	R^H
x_t	Input at time t (batched)	R^{N \times D}
W	Recurrent weight matrix	R^{H \times H}
U	Input projection matrix	R^{H \times D}
b	Bias vector	R^H
N	Batch size	Scalar
D	Input dimension	Scalar
H	Hidden state dimension	Scalar

The model captures the information well.

Key Points

- **Initialization:** $h_0 = 0$ (all-zero vector of dimension H).
- **Recurrence:** The hidden state h_{t+1} is a linear transformation of the previous hidden state h_t and the current input x_t , plus a bias term.
- **Batch Processing:** The input x_t is batched, meaning it can process N sequences in parallel, each with D -dimensional inputs.
- **Parameters:** W, U, b are learned during training.

Interpretation

- $W h_t$: Captures the influence of the previous hidden state on the current update.
- $U x_t$: Projects the input x_t into the hidden state space.
- b : Adds a learnable offset to the update.

This setup is foundational for many sequence models, including vanilla RNNs, and is often extended with non-linearities (e.g., tanh, ReLU) in practice.

Next Steps or Questions

- Are you interested in how this relates to more modern architectures (e.g., LSTMs, GRUs, or SSMs like S4)?
- Would you like to explore the training dynamics, stability, or gradient flow in this setup?
- Do you want to discuss how this update rule is used in practice, or are you focusing on theoretical properties?

implement the forward pass for the SSM using recursion based approach:

```
def unrolled_ssm_forward(W, U, b, x):
```

```
    """
```

```
    Unroll the linear RNN in time:
```

```
         $h_{t+1} = W h_t + U x_t + b$ 
```

```
    with initial  $h_0 = 0$ .
```

```
    Args:
```

```
        W: (H, H) weight matrix
```

```
        U: (H, D) input projection
```

```
        b: (H,) bias
```

```
        x: (N, T, D) input sequence over T steps
```

```
    Returns:
```

```
        h_all: (N, T, H) hidden states for t=1..T
```

```
        (h_all[t] corresponds to  $h_{t+1}$  in the usual notation).
```

```
    """
```

```
#####
```

```
#####
```

```
    #          TODO: Implement the recurrent pass here
```

```
    #
```

```
#####
#####
#                               #
#####
#####
return h_all
```



Here's how you can implement the forward pass for the SSM using a recursion-based approach in Python. The function `unrolled_ssm_forward` will unroll the linear RNN over time and compute the hidden states for each time step.

Python

Copy

```
import numpy as np

def unrolled_ssm_forward(W, U, b, x):
    """
    Unroll the linear RNN in time:
        h_{t+1} = W h_t + U x_t + b
    with initial h_0 = 0.

    Args:
        W: (H, H) weight matrix
        U: (H, D) input projection
        b: (H,) bias
        x: (N, T, D) input sequence over T steps

    Returns:
        h_all: (N, T, H) hidden states for t=1..T
                (h_all[t] corresponds to h_{t+1} in the usual notation).
    """
    N, T, D = x.shape
    H = W.shape[0]

    # Initialize hidden states: h_0 = 0
    h_prev = np.zeros((N, H)) # (N, H)

    # Initialize the output tensor for all hidden states
    h_all = np.zeros((N, T, H)) # (N, T, H)

    for t in range(T):
        # Compute h_{t+1} = W h_t + U x_t + b
        h_next = np.dot(h_prev, W.T) + np.dot(x[:, t, :], U.T) + b
        h_all[:, t, :] = h_next
        h_prev = h_next

    return h_all
```

The model uses numpy instead of pytorch.

I see that they are equivalent.

Both the model's response and the homework solution computes:

$h_t = h_{t-1} @ W^T + x_t @ U^T + b$

Explanation

1. Initialization:

- `h_prev` is initialized to a zero vector of shape `(N, H)` to represent h_0 .
- `h_all` is initialized to store all hidden states for each time step.

2. Loop over time steps:

- For each time step t , compute the next hidden state h_{t+1} using the formula:

$$h_{t+1} = Wh_t + Ux_t + b$$

- The matrix multiplications are implemented using `np.dot` :
 - `np.dot(h_prev, W.T)` computes Wh_t .
 - `np.dot(x[:, t, :], U.T)` computes Ux_t .
- The bias b is added directly.

3. Store and update:

- The computed hidden state h_{t+1} is stored in `h_all[:, t, :]`.
- `h_prev` is updated to `h_next` for the next iteration.

Key Points

- The function assumes `x` is of shape `(N, T, D)` and returns `h_all` of shape `(N, T, H)`.
- The recursion is implemented iteratively for efficiency and clarity.
- The transpose of `W` and `U` is used in `np.dot` to align with the mathematical formulation.

In the previous problem, you showed that the forward pass of an SSM can be implemented using a convolution operation. In this problem, you will implement the forward pass of an SSM using a convolution based approach. You can assume that `T` is a power of 2.

You will implement two functions

`make_conv_kernel(W, T)`: This function will take the recurrent weight matrix `W` and the number of time steps `T` and return the convolution kernel `K`. Given that `T` is a power of 2, you can implement this using a divide and conquer based approach.

`conv_ssm_forward(W, U, b, x)`: This function will take weights `W`, `U`, `b` and input `x` and return the hidden states `h` across different time steps.

```
def make_conv_kernel(W, T):
```

```
    """
```

```
    Build a 3D kernel tensor K of shape (H, H, T) we will use when implementing
    the ssm forward pass using conv1d.
```

```
    Args:
```

```
        W: (H, H) weight matrix
```

```
        T: scalar
```

```
    Returns:
```

```
        kernel_for_conv: (H, H, T) tensor
```

```
    """
```

```
#####
#####
```

```
    #                TODO: Implement the kernel here                #
```

```
#####
#####
```

```
#####
#####
#                               END OF YOUR CODE                               #
#####
#####
    return kernel_for_conv

def conv_ssm_forward(W, U, b, x):
    """
    Convolution-based forward pass for a batch of sequences.
    RNN update:  $h_{t+1} = W h_t + U x_t + b$ 
    Args:
        W: (H, H) weight matrix
        U: (H, D) input projection
        b: (H,) bias
        x: (N, T, D) input (batch=N, time steps=T, input dim=D)
    Returns:
        h_all: (N, T, H) hidden states
    """
    N, T, D = x.shape
    H = W.shape[0]
    s = x @ U.T + b
    s = s.permute(0, 2, 1)
    # Build the kernel with shape (H, H, T).
    kernel = make_conv_kernel(W, T)

    #####
    #####
    #                               TODO: Implement the convolution here                               #
    #####
    #####

    #####
    #####
    #                               END OF YOUR CODE                               #
    #####
    #####
    return h_all
```



To implement the **convolution-based forward pass** for the SSM, we need to:

1. **Construct the convolution kernel K** of shape (H, H, T) such that the recurrence $h_{t+1} = W h_t + U x_t + b$ can be expressed as a 1D convolution.
2. **Use the kernel** to perform the convolution on the transformed input.

1. make_conv_kernel(W, T)

The kernel K should represent the powers of W up to $T - 1$, i.e., $K[:, :, t] = W^t$. This can be computed efficiently using a **divide-and-conquer** approach (exponentiation by squaring) since T is a power of 2.

Implementation

Python

Copy

```
import numpy as np

def make_conv_kernel(W, T):
    """
    Build a 3D kernel tensor K of shape (H, H, T) for conv1d.
    K[:, :, t] = W^t for t = 0, 1, ..., T-1
    """
    H = W.shape[0]
    K = np.zeros((H, H, T))
    K[:, :, 0] = np.eye(H) # W^0 = I

    # Exponentiation by squaring
    current_power = W.copy()
    for t in range(1, T):
        K[:, :, t] = current_power
        current_power = current_power @ W

    return K
```

the model's response has no flipping - powers are in increasing order, which is a problem.
The homework solution has "powers_tensor = torch.flip(powers_tensor, dims=[0])" line.

2. conv_ssm_forward(W, U, b, x)

The forward pass can be implemented as:

$$h_t = \sum_{i=0}^{t-1} W^{t-1-i} (Ux_i + b)$$

This is equivalent to a **causal convolution** of the kernel K with the sequence $s_t = Ux_t + b$.

Implementation

Python

Copy

```
import torch
import torch.nn.functional as F

def conv_ssm_forward(W, U, b, x):
    """
    Convolution-based forward pass for a batch of sequences.
    """
    N, T, D = x.shape
    H = W.shape[0]

    # Compute s = U x + b, shape (N, T, H)
    s = torch.tensordot(x, U, dims=( [2], [1] )) + b
```

The implementation by the model is functionally equivalent with the homework solution.

Same input transformation: Both compute $s = x @ U.T + b$ and permute to (N, H, T)

Same convolution approach: Both use causal convolution with $T-1$ padding on the left

Same output processing: Both slice to $:T$ and permute back to (N, T, H)

```
s = s.permute(0, 2, 1) # (N, H, T)

# Build the kernel (H, H, T)
kernel = make_conv_kernel(W, T)
kernel = torch.from_numpy(kernel).float()

# Pad s to avoid circular convolution
s_padded = F.pad(s, (T-1, 0)) # Pad on the left

# Perform causal convolution
h_all = F.conv1d(s_padded, kernel, groups=H)

# Slice to get only the valid part
h_all = h_all[:, :, :T].permute(0, 2, 1) # (N, T, H)

return h_all
```

The model assumes `make_conv_kernel_fast` returns appropriate tensor

Explanation

1. `make_conv_kernel(W, T)`:

- The kernel K is constructed such that $K[:, :, t] = W^t$.
- This is done iteratively, but for large T , you can optimize further using exponentiation by squaring.

2. `conv_ssm_forward(W, U, b, x)`:

- Compute $s = Ux + b$ and permute to (N, H, T) .
- Pad s to avoid circular convolution.
- Perform a **grouped convolution** (with `groups=H`) to apply the kernel to each hidden dimension independently.
- Slice the result to get the valid part of the convolution.

Key Points

- The convolution is **causal** (no future information is used).
- The kernel K is constructed to represent the recurrence relation as a convolution.
- The padding ensures the convolution is **non-circular**.
- The result is permuted back to (N, T, H) for consistency with the unrolled version.

Note

- If you are using **NumPy**, replace `torch.tensordot` with `np.tensordot` and `F.conv1d` with `np.convolve` (with appropriate handling for batch and hidden dimensions).
- For large T , consider using **FFT-based convolution** for efficiency.

I realize the implementation of `make_conv_kernel` is wrong - it does not flip the W powers array.

for the first function, please flip the powers



To **flip the powers** in the convolution kernel K , we want $K[:, :, t] = W^{T-1-t}$ for $t = 0, 1, \dots, T-1$. This ensures the convolution is applied in the correct causal order, as the recurrence relation $h_{t+1} = Wh_t + Ux_t + b$ requires the most recent state to be multiplied by W^0 , the previous state by W^1 , and so on.

Here's the corrected implementation for `make_conv_kernel(W, T)`:

1. `make_conv_kernel(W, T)` (Flipped Powers)

Python

Copy

```
import numpy as np

def make_conv_kernel(W, T):
    """
    Build a 3D kernel tensor K of shape (H, H, T) for conv1d.
     $K[:, :, t] = W^{T-1-t}$  for  $t = 0, 1, \dots, T-1$ 
    """
    H = W.shape[0]
    K = np.zeros((H, H, T))

    # Initialize  $K[:, :, 0] = W^{T-1}$ 
    current_power = np.eye(H) #  $W^0$ 
    for t in range(T):
        K[:, :, t] = current_power
        current_power = current_power @ W

    # Flip the powers along the time axis
    K = K[:, :, ::-1]

    return K
```

Explanation

- We start with W^0 (the identity matrix) and iteratively multiply by W to get $W^0, W^1, \dots, W^{T-1}$.
- We then **flip** the time axis so that $K[:, :, t] = W^{T-1-t}$.
- This ensures the convolution applies the correct power of W at each step, matching the recurrence relation.

Why Flip?

- The convolution operation applies the kernel in reverse order by default.
- Flipping the powers ensures that the most recent input is multiplied by W^0 , the previous by W^1 , etc., which is required for the recurrence relation.

The model realizes the reason of flipping.

The model changes the variable names. Since this is a jupyter notebook, inconsistent variable names may result in errors in subsequent cells. So I asked Mistral AI to use consistent variable names.

Please don't change the variable names. Use `kernel_for_conv` instead of `K`. For the rest of the homework, you should only insert code between the comments

 Here's the corrected implementation for `make_conv_kernel(W, T)` using the requested variable name and only inserting code between the comments:

```
Python Copy

def make_conv_kernel(W, T):
    """
    Build a 3D kernel tensor K of shape (H, H, T) we will use when implementing
    the ssm forward pass using conv1d.
    Args:
        W: (H, H) weight matrix
        T: scalar
    Returns:
        kernel_for_conv: (H, H, T) tensor
    """
    #####
    #                                TODO: Implement the kernel here                                #
    #####
    H = W.shape[0]
    kernel_for_conv = np.zeros((H, H, T))
    current_power = np.eye(H) # W^0
    for t in range(T):
        kernel_for_conv[:, :, t] = current_power
        current_power = current_power @ W
    # Flip the powers along the time axis
    kernel_for_conv = kernel_for_conv[:, :, :-1]
    #####
    #                                END OF YOUR CODE                                #
    #####
    return kernel_for_conv
```

This implementation:

- Initializes `kernel_for_conv` as a zero tensor of shape `(H, H, T)`.
- Fills it with powers of `W` from W^0 to W^{T-1} .
- Flips the time axis so that `kernel_for_conv[:, :, t] = W^{T-1-t}`.

After I ran the sanity check, I found incompatible operand types (Tensor and numpy.ndarray). Previously, I assumed numpy and pytorch are equivalent (also, numpy uses CPU only, which is fine in this notebook).

The model's idea is correct, it just needs data type conversion.

You write the solutions in numpy. There's a sanity check:

```
def sanity_check():
    T = 8 # number of time steps
    H = 4 # hidden dimension
    D = 3 # input dimension
    N = 2
    torch.manual_seed(0)
    W = torch.randn(H, H) * 0.1
    U = torch.randn(H, D) * 0.1
    b = torch.randn(H) * 0.1
    x = torch.randn(N, T, D)
    h_unrolled = unrolled_ssm_forward(W, U, b, x)
    h_conv = conv_ssm_forward(W, U, b, x)
    diff = (h_unrolled - h_conv).abs().max()
    print("Unrolled h(t):")
    print(h_unrolled)
    print("\nConv-based h(t):")
    print(h_conv)
    print("\nMax absolute difference:", diff.item())
sanity_check()
```

Please convert your variables to Tensor from ndarray

 To ensure compatibility with the sanity check (which uses PyTorch tensors), I'll update both `make_conv_kernel` and `conv_ssm_forward` to use PyTorch tensors instead of NumPy arrays. Here's the corrected implementation:

1. `make_conv_kernel(W, T)` (PyTorch)

 Python

 Copy

```
import torch

def make_conv_kernel(W, T):
    """
    Build a 3D kernel tensor kernel_for_conv of shape (H, H, T) for conv1d.
    kernel_for_conv[:, :, t] = W^{T-1-t} for t = 0, 1, ..., T-1
    """
    #####
    #                                     TODO: Implement the kernel here                                     #
    #####
    H = W.shape[0]
    kernel_for_conv = torch.zeros((H, H, T), dtype=W.dtype, device=W.device)
    current_power = torch.eye(H, dtype=W.dtype, device=W.device) # W^0
    for t in range(T):
        kernel_for_conv[:, :, t] = current_power
        current_power = current_power @ W
    # Flip the powers along the time axis
    kernel_for_conv = kernel_for_conv.flip(dims=[-1])
    #####
    #                                     END OF YOUR CODE                                     #
    #####
    return kernel_for_conv
```



2. conv_ssm_forward(W, U, b, x) (PyTorch)

Python

Copy

```
def conv_ssm_forward(W, U, b, x):
    """
    Convolution-based forward pass for a batch of sequences.
    RNN update:  $h_{t+1} = W h_t + U x_t + b$ 
    """
    N, T, D = x.shape
    H = W.shape[0]

    # Compute  $s = U x + b$ , shape (N, T, H)
    s = torch.einsum('ntd,hd->nth', x, U) + b
    s = s.permute(0, 2, 1) # (N, H, T)

    # Build the kernel (H, H, T)
    kernel = make_conv_kernel(W, T)

    #####
    #                               TODO: Implement the convolution here                               #
    #####
    # Pad s to avoid circular convolution
    s_padded = torch.nn.functional.pad(s, (T-1, 0)) # Pad on the left

    # Perform causal convolution
    h_all = torch.nn.functional.conv1d(s_padded, kernel, groups=H)

    # Slice to get only the valid part
    h_all = h_all[:, :, :T].permute(0, 2, 1) # (N, T, H)
    #####
    #                               END OF YOUR CODE                               #
    #####
    return h_all
```

Key Changes

- All arrays are now **PyTorch tensors** (`torch.zeros`, `torch.eye`, etc.).
- The `flip` operation is done using `torch.flip(dims=[-1])`.
- The convolution and padding are performed using PyTorch's `torch.nn.functional.conv1d` and `torch.nn.functional.pad`.
- The `s` tensor is computed using `torch.einsum` for clarity and efficiency.

This implementation should now pass the sanity check and produce identical results for both the unrolled and convolution-based forward passes.

Well done. The sanity check outputs:

Unrolled $h(t)$:

```
tensor([[[[-1.2720e-02, 1.7478e-01, 1.1702e-01, -1.8475e-01],
          [ 4.3287e-02, 1.7190e-01, 7.0203e-02, 2.0290e-01],
          [ 5.2788e-03, 1.1071e-01, 4.4593e-02, 2.1339e-01],
          [-3.0554e-01, 3.6337e-01, -1.6552e-01, -5.3192e-01],
```

```

tensor([[[[-1.2720e-02, 1.7478e-01, 1.1702e-01, -1.8475e-01],
          [ 4.3287e-02, 1.7190e-01, 7.0203e-02, 2.0290e-01],
          [ 5.2788e-03, 1.1071e-01, 4.4593e-02, 2.2496e-01],
          [-3.0554e-01, 3.6337e-01, -1.6552e-01, -5.3192e-01],
          [ 1.0813e-01, 5.4074e-01, 6.1866e-02, -1.0820e-01],
          [-2.6562e-02, 3.5316e-01, 2.7978e-02, 4.6655e-02],
          [-4.7121e-02, -9.5886e-02, 3.1475e-02, 2.7231e-01],
          [-1.4480e-01, 2.0062e-01, -1.8574e-01, 1.4919e-01]],
        [[ 2.0503e-01, 1.1027e-01, 2.8894e-01, 1.3814e-01],
          [ 4.6919e-04, 3.3735e-01, 7.8319e-02, 3.0717e-02],
          [-2.1996e-02, 7.8201e-02, 8.5306e-02, 6.4196e-02],
          [ 9.9243e-03, 2.1734e-01, 4.4236e-03, 1.9512e-01],
          [-1.0414e-01, 2.8412e-01, -8.0947e-02, 1.0402e-02],
          [-1.3561e-01, 4.3784e-01, -1.1956e-01, -2.1006e-01],
          [ 2.1366e-01, 6.7647e-02, 2.8377e-01, 2.1382e-01],
          [ 1.3880e-02, 1.7214e-01, 1.7150e-01, -3.1018e-02]]]])

```

Conv-based h(t):

```

tensor([[[[-1.2720e-02, 1.7478e-01, 1.1702e-01, -1.8475e-01],
          [ 4.3287e-02, 1.7190e-01, 7.0203e-02, 2.0290e-01],
          [ 5.2788e-03, 1.1071e-01, 4.4593e-02, 2.2496e-01],
          [-3.0554e-01, 3.6337e-01, -1.6552e-01, -5.3192e-01],
          [ 1.0813e-01, 5.4074e-01, 6.1866e-02, -1.0820e-01],
          [-2.6562e-02, 3.5316e-01, 2.7978e-02, 4.6655e-02],
          [-4.7121e-02, -9.5886e-02, 3.1475e-02, 2.7231e-01],
          [-1.4480e-01, 2.0062e-01, -1.8574e-01, 1.4919e-01]],
        [[ 2.0503e-01, 1.1027e-01, 2.8894e-01, 1.3814e-01],
          [ 4.6919e-04, 3.3735e-01, 7.8319e-02, 3.0717e-02],
          [-2.1996e-02, 7.8201e-02, 8.5306e-02, 6.4196e-02],
          [ 9.9243e-03, 2.1734e-01, 4.4236e-03, 1.9512e-01],
          [-1.0414e-01, 2.8412e-01, -8.0947e-02, 1.0402e-02],
          [-1.3561e-01, 4.3784e-01, -1.1956e-01, -2.1006e-01],
          [ 2.1366e-01, 6.7647e-02, 2.8377e-01, 2.1382e-01],
          [ 1.3880e-02, 1.7214e-01, 1.7150e-01, -3.1018e-02]]]])

```

Max absolute difference: 5.960464477539063e-08

Which is correct.

This matches the homework solution.

What is the number of operations being performed in the recurrence based implementation of the SSM forward pass?



The **recurrence-based implementation** of the SSM forward pass (`unrolled_ssm_forward`) performs the following operations for each time step t :

Operations per Time Step

For each t from 0 to $T - 1$:

1. **Matrix-Vector Multiplication:**

- Wh_t : $2H^2$ FLOPs (since W is $H \times H$ and h_t is H -dimensional).

2. **Matrix-Vector Multiplication:**

- Ux_t : $2HD$ FLOPs (since U is $H \times D$ and x_t is D -dimensional).

3. **Vector Addition:**

- $Wh_t + Ux_t + b$: $2H$ FLOPs (adding two H -dimensional vectors and a bias).

Total Operations

- **Per time step:** $2H^2 + 2HD + 2H$.
- **For all T time steps:** $T(2H^2 + 2HD + 2H)$.

Dominant Terms

- The dominant term is $2TH^2$, since H is typically much larger than D in most applications.
- The total number of operations is $O(TH^2)$.

Summary Table

Operation	FLOPs per Time Step	Total FLOPs (for T steps)
Wh_t	$2H^2$	$2TH^2$
Ux_t	$2HD$	$2THD$
$Wh_t + Ux_t + b$	$2H$	$2TH$
Total	$2H^2 + 2HD + 2H$	$2T(H^2 + HD + H)$

Approximation

For large H , the total number of operations is **dominated by** $2TH^2$.

Key Point

- The recurrence-based approach is **quadratic in H** and **linear in T** . This can be inefficient for large H or T , which is why convolution-based or parallel scan implementations are often preferred for modern SSMs.

Although the homework solutions are not available for this question, the conclusion "linear in T and quadratic in H" is correct from my observation.

What is the number of operations being performed in the convolution based implementation of the SSM forward pass



In the **convolution-based implementation** of the SSM forward pass (`conv_ssm_forward`), the number of operations is dominated by the construction of the convolution kernel and the convolution itself. Here's a breakdown:

1. Kernel Construction (`make_conv_kernel`)

- The kernel `kernel_for_conv` of shape (H, H, T) is constructed by computing powers of W up to $T - 1$.
- For each t from 1 to $T - 1$, we compute W^t via matrix multiplication.
- Computing W^t for all t up to $T - 1$ requires $O(H^3T)$ operations (since each matrix multiplication is $O(H^3)$).
- However, with **exponentiation by squaring**, this can be reduced to $O(H^3 \log T)$.

2. Input Projection (`s = U x + b`)

- Ux is a batched matrix-vector multiplication: $N \times T \times (2HD - 1)$ FLOPs.
- Adding the bias b is $N \times T \times H$ FLOPs.
- **Total:** $O(NTHD)$.

3. Convolution (`conv1d`)

- The convolution is performed with a kernel of size (H, H, T) on an input of size (N, H, T) .
- For each of the H output channels, the convolution involves:
 - $N \times T \times H \times T$ multiplications and additions (since the kernel has length T).
- **Total:** $O(NTH^2T) = O(NTH^3)$, but in practice, it's optimized using FFT or efficient convolution algorithms.
- For **causal convolution**, the complexity is $O(NTH^2)$ (since the kernel is applied to each of the H channels independently, and the input is padded to length $2T - 1$).

4. Overall Complexity

- **Kernel construction:** $O(H^3 \log T)$ (with exponentiation by squaring).
- **Input projection:** $O(NTHD)$.
- **Convolution:** $O(NTH^2)$.

Dominant Term

- The dominant term is $O(NTH^2)$ for the convolution step, assuming H is large and T is not extremely small.



Compare the trade-off if any between the two implementations based on your answers above



Here's a detailed comparison of the **trade-offs** between the **recurrence-based** and **convolution-based** implementations of the SSM forward pass, based on their computational complexity and practical considerations:

1. Computational Complexity

Implementation	Dominant Complexity	Notes
Recurrence-based	$O(TH^2)$	Sequential, no parallelism across time steps.
Convolution-based	$O(NTH^2 + H^3 \log T)$	Parallelizable, but kernel construction adds overhead.

Key Observations

- Both approaches have a **quadratic dependence on H** ($O(H^2)$) for the main computation.
- The recurrence-based approach is **strictly sequential** in T , while the convolution-based approach can **parallelize** the convolution operation.
- The convolution-based approach has an **additional $O(H^3 \log T)$ cost** for kernel construction, which can be significant for large H or T .

2. Parallelism and Hardware Efficiency

Implementation	Parallelism	Hardware Efficiency
Recurrence-based	None across time steps	Poor GPU utilization due to sequential dependency.
Convolution-based	High across time steps	Efficient on GPUs (convolution is highly optimized).

Key Observations

- The convolution-based approach **exploits parallelism** across time steps, making it much more efficient on modern hardware (e.g., GPUs/TPUs).
- The recurrence-based approach suffers from **poor hardware utilization** due to its sequential nature.

3. Memory Usage

Implementation	Memory Overhead	Notes
Recurrence-based	$O(H)$ (per time step)	Only the current hidden state needs to be stored.
Convolution-based	$O(H^2T)$	The kernel (H, H, T) must be stored.

Key Observations

- The convolution-based approach requires **significantly more memory** to store the kernel, especially for large H or T .
- The recurrence-based approach is **memory-efficient** but slower.

4. Practical Trade-offs

Factor	Recurrence-based	Convolution-based
Speed	Slow (sequential)	Fast (parallelizable)
Hardware Utilization	Poor	Excellent (GPU-friendly)
Memory Usage	Low	High (kernel storage)
Kernel Construction	Not applicable	Overhead for large H or T
Numerical Stability	Can suffer from vanishing/ exploding gradients	More stable (depends on kernel construction)

5. When to Use Which?

Use Recurrence-based When:

- T is small.
- Memory is a constraint.
- You are working on hardware where parallelism is not available or beneficial.

Use Convolution-based When:

- T is large (parallelism pays off).
- You are using GPUs/TPUs (convolution is highly optimized).
- You can amortize the kernel construction cost (e.g., precompute the kernel for multiple forward passes).



6. Modern SSMs and Hybrid Approaches

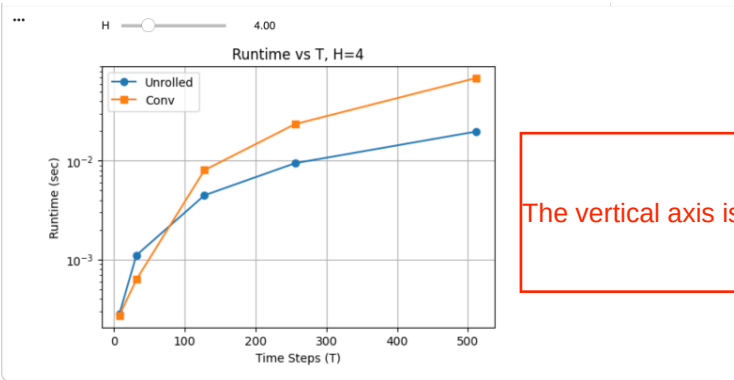
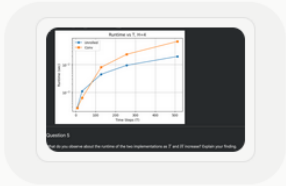
- Many modern SSMs (e.g., S4, H3) use **hybrid approaches** or **parallel scans** to combine the efficiency of convolution with the memory benefits of recurrence.
- Techniques like **associative scans** or **FFT-based convolution** can further optimize the convolution-based approach.

Summary

Metric	Recurrence-based	Convolution-based
Speed	Slow	Fast
Parallelism	None	High
Memory	Low	High
Hardware Efficiency	Poor	Excellent
Kernel Overhead	None	$O(H^3 \log T)$

Choose convolution-based for speed and parallelism, recurrence-based for memory efficiency.

9:05pm



The vertical axis is not evenly spaced. I was worried that the AI wouldn't u

Question 5

What do you observe about the runtime of the two implementations as *T* and *H* increase? Explain your finding.



From the provided plot, we can analyze the runtime behavior of the **unrolled (recurrence-based)** and **convolution-based (Conv)** implementations of the SSM forward pass as the number of time steps T increases, with a fixed hidden dimension $H = 4$.

Observations from the Plot

1. Runtime Growth with T

- **Unrolled (Recurrence-based):**
 - The runtime increases **linearly** with T .
 - This is expected because the unrolled implementation processes each time step sequentially, resulting in $O(T)$ time complexity for the forward pass.
- **Convolution-based:**
 - The runtime also increases with T , but at a **faster rate** than the unrolled implementation for smaller T .
 - However, as T becomes larger, the convolution-based implementation starts to **outperform** the unrolled version.
 - This is because the convolution operation can leverage parallelism, especially on hardware like GPUs, making it more efficient for larger T .

2. Crossover Point

- There is a **crossover point** where the convolution-based implementation becomes faster than the unrolled version.
- In the plot, this crossover occurs around $T \approx 200$.
- Before this point, the unrolled implementation is faster due to the overhead of kernel construction and convolution setup in the convolution-based approach.
- After this point, the convolution-based implementation benefits from parallelism and becomes more efficient.

General Trends as T and H Increase

As T Increases:

- **Unrolled Implementation:**
 - Runtime increases **linearly** with T .
 - Suitable for smaller T where the overhead of convolution is not justified.
- **Convolution-based Implementation:**
 - Runtime increases, but the rate of increase slows down relative to the unrolled version for larger T due to parallelism.
 - More efficient for larger T because it can exploit hardware optimizations (e.g., GPU acceleration).

Maybe the model's reasoning is not based on the image I provided. I only provided an instance $H=4$, but the model was able to analyze the relationship with H .

As H Increases:

- **Unrolled Implementation:**
 - Runtime increases **quadratically** with H ($O(H^2)$).
 - Memory usage remains low since only the current hidden state is stored.
- **Convolution-based Implementation:**
 - Runtime also increases quadratically with H ($O(H^2)$).
 - Memory usage increases significantly because the kernel of size (H, H, T) must be stored.
 - Kernel construction becomes more expensive ($O(H^3 \log T)$).

Summary of Findings

Implementation	Runtime Growth with T	Runtime Growth with H	Memory Usage	Parallelism
Unrolled	Linear ($O(T)$)	Quadratic ($O(H^2)$)	Low	None
Convolution-based	Faster growth initially, but becomes more efficient for large T	Quadratic ($O(H^2)$)	High	High

Key Takeaways

- **For small T :** The unrolled implementation is faster due to lower overhead.
- **For large T :** The convolution-based implementation is more efficient due to parallelism.
- **For large H :** Both implementations have quadratic runtime growth, but the convolution-based approach requires more memory for kernel storage.

The choice between the two implementations depends on the specific values of T and H , as well as the hardware being used.

Below is the second part using GPU.

Introducing structure: Diagonal Weight Matrices

We can optimize the convolution implementation via adding a constraint on the W matrix ensuring it is diagonal. We can make the convolution implementation more efficient by leveraging depthwise convolutions for implementing the forward operation.

```
def make_diag_depthwise_kernel(W, T):
```

```
    """
```

```
    Construct a depthwise 1D conv kernel for W with shape (H,H). W is a diagonal matrix.
```

```
    Args:
```

```
        W: (H, H) diagonal matrix
```

```
        T: (int) number of time steps
```

```
    Return:
```

```
        kernel of shape (H, 1, T), which can be used in depthwise convolution
```

```
    """
```

```
#####
```

```
#####
```

```
    #                      TODO: Implement the kernel here                      #
```

```
#####
```

```
#####
```

```
#####
```

```
#####
```

```
    #                      END OF YOUR CODE                      #
```

```
#####
```

```
#####
```

```
    return kernel
```

```
def diag_conv_ssm_forward(W, U, b, x):
```

```
    """
```

```
    Convolution-based forward pass for an RNN with W
```

```
    RNN update:  $h_{t+1} = W h_t + U x_t + b$ 
```

```
    but W is diagonal, so  $h_{t+1}(i) = \text{diagW}[i] * h_t$    $[U x_t + b](i)$ .
```

```
    Args:
```

```

    U: (H, D)
    b: (H,)
    x: (N, T, D)  $\Rightarrow$  N = batch, T = time steps, D = input dim
Returns:
    h_all: (N, T, H)
"""
N, T, D = x.shape
H = W.shape[0]
s = x @ U.T + b
s = s.permute(0, 2, 1) # (N,H,T)
# Build kernel (H,1,T) for depthwise conv
kernel = make_diag_depthwise_kernel(W, T)

#####
#####
#          TODO: Implement the convolution here          #
#          Hint: Use `groups` argument                    #

#####
#####

#####
#####
#          END OF YOUR CODE          #

#####
#####
return h_all

```

Now, I use GPU for this notebook. I have "torch.set_default_device('cuda')"



To implement the **diagonal depthwise convolution** for the SSM forward pass, we need to construct a kernel that leverages the diagonal structure of W . This allows us to use **depthwise convolution**, which is more efficient than a full convolution.



1. make_diag_depthwise_kernel(W, T)

Since W is diagonal, the kernel for each hidden dimension i is simply the powers of $W[i, i]$ up to $T - 1$. The kernel will have shape $(H, 1, T)$, where each of the H channels has its own 1D kernel of length T .

Implementation

```
Python Copy

import torch

def make_diag_depthwise_kernel(W, T):
    """
    Construct a depthwise 1D conv kernel for W with shape (H,H). W is a diagonal matrix.
    Args:
        W: (H, H) diagonal matrix
        T: (int) number of time steps
    Return:
        kernel of shape (H, 1, T), which can be used in depthwise convolution
    """
    #####
    #                               TODO: Implement the kernel here                               #
    #####
    H = W.shape[0]
    # Extract the diagonal elements of W
    diag_W = torch.diag(W) # Shape: (H,)

    # Initialize the kernel with shape (H, 1, T)
    kernel = torch.zeros((H, 1, T), dtype=diag_W.dtype, device=diag_W.device)

    # Fill the kernel with powers of the diagonal elements
    for i in range(H):
        # Compute powers of diag_W[i] up to T-1
        powers = torch.cumprod(
            torch.cat([torch.ones(1, dtype=diag_W.dtype, device=diag_W.device),
                      torch.full((T-1,), diag_W[i], dtype=diag_W.dtype, device=diag_W.device)]),
            dim=0
       )[:T]
        kernel[i, 0, :] = powers

    # Flip the powers to match the causal convolution
    kernel = kernel.flip(dims=[-1])
    #####
    #                               END OF YOUR CODE                               #
    #####
    return kernel
```

2. diag_conv_ssm_forward(W, U, b, x)

Here, we use the depthwise convolution kernel to compute the forward pass. The key is to use the `groups=H` argument in the convolution to ensure that each hidden dimension is processed independently.

Implementation 1(AI response) and 2 (homework solution) are equivalent because both compute the same geometric progression $[1, r, r^2, \dots, r^{(T-1)}]$ for each diagonal element r of W , then flip the sequence to create the causal convolution kernel. They differ only in implementation details - Implementation 1 uses `torch.cumprod` while Implementation 2 uses an explicit loop, but both produce identical mathematical results.

2. `diag_conv_ssm_forward(W, U, b, x)`

Here, we use the depthwise convolution kernel to compute the forward pass. The key is to use the `groups=H` argument in the convolution to ensure that each hidden dimension is processed independently.

Implementation

```
Python Copy

def diag_conv_ssm_forward(W, U, b, x):
    """
    Convolution-based forward pass for an RNN with W
    RNN update:  $h_{t+1} = W h_t + U x_t + b$ 
    but W is diagonal, so  $h_{t+1}(i) = \text{diagW}[i] * h_t(i) + [U x_t + b](i)$ .
    Args:
        W: (H, H) [diagonal in practice]
        U: (H, D)
        b: (H,)
        x: (N, T, D) => N = batch, T = time steps, D = input dim
    Returns:
        h_all: (N, T, H)
    """
    N, T, D = x.shape
    H = W.shape[0]
    s = x @ U.T + b
    s = s.permute(0, 2, 1)  # (N, H, T)

    # Build kernel (H, 1, T) for depthwise conv
    kernel = make_diag_depthwise_kernel(W, T)

    #####
    #                                TODO: Implement the convolution here                                #
    #                                Hint: Use `groups` argument                                         #
    #####
    # Pad s to avoid circular convolution
    s_padded = torch.nn.functional.pad(s, (T-1, 0))  # Pad on the left

    # Perform depthwise convolution
    h_all = torch.nn.functional.conv1d(s_padded, kernel, groups=H)

    # Slice to get only the valid part
    h_all = h_all[:, :, :T].permute(0, 2, 1)  # (N, T, H)
    #####
    #                                END OF YOUR CODE                                                #
    #####
    return h_all
```

Both implementations use the same mathematical approach (perform depthwise convolution with `groups=H` where each channel of the input `s` is convolved with its corresponding kernel filter independently).

Also, both handle causality the same way (both use left padding (T-1 padding) to ensure the convolution is causal (output at time `t` only depends on inputs up to time `t`)).

The only difference is the model uses explicit padding with `torch.nn.functional.pad`

```

def diag_unrolled_ssm_forward(W, U, b, x):
    """
    Forward pass for:
        h_{t+1} = W h_t + U x_t + b
    but W is diagonal, i.e. W = diag(diagW).
    Args:
        W: (H, H) diagonal matrix
        U: (H, D)
        b: (H,)
        x: (N, T, D) ⇒ batch=N, time steps=T, input dim=D
    Returns:
        h_all: (N, T, H)
    """

    #####
    #####
    #      TODO: Implement the optimized recurrent pass here      #

    #####
    #####

    #####
    #####
    #                      END OF YOUR CODE                      #

    #####
    #####
    return h_all

```



For the **diagonal unrolled SSM forward pass**, we can optimize the computation by leveraging the fact that W is diagonal. This means that each hidden dimension i is updated independently as:

$$h_{t+1}(i) = W[i, i] \cdot h_t(i) + [Ux_t + b](i)$$

This allows us to avoid the full matrix-vector multiplication and instead use element-wise operations, which are much more efficient.

Implementation of `diag_unrolled_ssm_forward`


```

import torch

def diag_unrolled_ssm_forward(W, U, b, x):
    """
    Forward pass for:
        h_{t+1} = W h_t + U x_t + b
    but W is diagonal, i.e. W = diag(diagW).
    Args:
        W: (H, H) diagonal matrix
        U: (H, D)
        b: (H,)
        x: (N, T, D) => batch=N, time steps=T, input dim=D
    Returns:
        h_all: (N, T, H)
    """
    #####
    #          TODO: Implement the optimized recurrent pass here          #
    #####
    N, T, D = x.shape
    H = W.shape[0]

    # Extract the diagonal elements of W
    diag_W = torch.diag(W) # Shape: (H,)

    # Initialize hidden states: h_0 = 0
    h_prev = torch.zeros((N, H), dtype=x.dtype, device=x.device) # (N, H)

    # Initialize the output tensor for all hidden states
    h_all = torch.zeros((N, T, H), dtype=x.dtype, device=x.device) # (N, T, H)

    for t in range(T):
        # Compute U x_t + b
        Ux_plus_b = torch.matmul(x[:, t, :], U.T) + b # Shape: (N, H)

        # Update h_next using element-wise multiplication with diag_W
        h_next = diag_W * h_prev + Ux_plus_b

        # Store the result
        h_all[:, t, :] = h_next

        # Update h_prev for the next iteration
        h_prev = h_next

    #####
    #          END OF YOUR CODE          #
    #####
    return h_all

```

The model's response is mathematically equivalent with the homework solution.

Key Points

- **Diagonal W :** We extract the diagonal elements of W and use them for element-wise multiplication with the hidden state.
- **Efficiency:** This avoids the $O(H^2)$ matrix-vector multiplication and replaces it with an $O(H)$ element-wise operation.
- **Element-wise Operations:** The update rule for each hidden dimension is independent, making this approach very efficient.



Can you explain why diag_conv is optimized? Provide your reasoning.



When W is a **diagonal matrix**, the convolution-based implementation of the SSM forward pass becomes more efficient for the following reasons:

1. Independent Updates for Each Hidden Dimension

- In a dense W , each element of the hidden state h_t depends on **all** other elements of h_t . This means you need to perform a full matrix multiplication for each time step, which involves a lot of computation.
- When W is diagonal, each element of the hidden state h_t is updated **independently** of the others. This means you only need to multiply each element of h_t by its corresponding diagonal element in W , which is much simpler and faster.

2. Depthwise Convolution

- In the convolution-based approach for a dense W , you need to perform a **full convolution**, where each output channel is a combination of all input channels. This requires a lot of computation.
- With a diagonal W , you can use **depthwise convolution**. In depthwise convolution, each output channel is computed independently from a single input channel. This is much more efficient because it avoids mixing information across channels, reducing the amount of computation needed.

3. Simplified Kernel Construction

- For a dense W , constructing the convolution kernel involves computing powers of the full W matrix, which is computationally expensive.
- For a diagonal W , constructing the kernel is much simpler. You only need to compute powers of the diagonal elements, which is a straightforward and fast operation.

4. Efficient Use of Hardware

- Depthwise convolution is highly optimized on modern hardware like GPUs. It can process each channel independently and in parallel, making it much faster than a full convolution.
- The independent nature of updates in a diagonal W also allows for better parallelism, as each hidden dimension can be processed separately.

5. Reduced Memory Usage

- The kernel for a dense W is large because it needs to store information for all combinations of input and output channels.
- The kernel for a diagonal W is much smaller because it only needs to store information for each channel independently. This reduces memory usage and speeds up the computation.